01 / PURPOSE & LEARNING ROUTE

Set up. Practise. Prove your work.

GitHub & development environment

Independent-learning handbook • Software, Cloud and Cybersecurity • Version 1.0 • 9 November 2026

This guide supports your existing WIL tasks. It is not a replacement programme, an extension of deadlines, or proof that work has been approved. You will install and check your own tools, practise version control, and explain your results.

Your route through the handbook

- **Pages 2–3:** prepare your computer and verify your tools.
- **Pages 4–6:** obtain private repository access and complete a GitHub workflow.
- **Page 7:** build and run your own small local exercise.
- **Page 8:** choose appropriate evidence for your stream.
- **Pages 9–10:** diagnose errors and submit a useful support request.
- **Page 11:** follow official documentation and check your understanding.

> **Working standard:** read first, attempt the step, verify the result, then record what you learned. If blocked, preserve your work and request targeted help. Never ask someone to complete assessed work for you.

Companion document

Use the separate Setup & Evidence Checklist while you work. Complete it with your own observations; a tick alone is not evidence.

02 / PREPARE SAFELY

Know your starting point.

Before you install anything

• Record your operating system and version, available storage, and whether you can install applications. Do not post device serial numbers or personal account details.

• Use your own computer or an approved university lab device. On a managed computer, ask the lab administrator before installing software or changing settings.

• Create a dedicated working folder, for example Documents/wil-lab. Keep it separate from ID documents, CVs and shared cloud-sync folders.

• Check that your browser and operating system are supported and updated. Download tools from official websites only.

Choose an appropriate setup

• **Windows/macOS:** VS Code, Git and a browser are sufficient for the first exercise. GitHub Desktop is an optional graphical Git client.

• **Linux:** use VS Code and command-line Git. GitHub Desktop has no official Linux distribution; this guide does not require unofficial builds.

• **Phone/Chromebook/restricted device:** browser editing can help with small documentation changes, but it does not prove a working local development environment. Record the limitation and ask about an approved lab computer.

> **No paid account is required.** Cloud provider access remains deferred. Do not enter billing details to solve a setup problem. Virtual machines, containers and browser-hosted development environments are optional unless your specific task requires them.

Private information stays outside your repository

Never commit certified IDs, student numbers, passwords, access tokens, private keys, real customer data or company credentials. Submit required identity documents only through the protected student profile process.

03 / INSTALL & VERIFY

Check tools, not just icons.

1. Install VS Code

Open code.visualstudio.com/Download, select your operating system and follow the installer. Open the editor, then File → Open Folder and choose your lab folder. Use Terminal → New Terminal for the checks below.

2. Install Git

Use git-scm.com/downloads. On Linux, follow the official instructions for your distribution. After installation, close and reopen your terminal and VS Code.

```
git --version
```

Pass: a Git version is displayed. **Fail:** “not recognised” or “command not found”; see page 9. Record the actual version, not a guessed value.

3. Set your commit identity

Use your own name and your chosen commit email. For privacy, copy the exact no-reply email shown under GitHub Settings → Emails. Replace the examples; do not copy another student’s identity.

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_COMMIT_EMAIL"
git config --get user.name
git config --get user.email
```

4. Add only the runtime your project needs

The HTML exercise on page 7 needs no Node.js. If your assigned project requires Node.js, install the current LTS release from nodejs.org/en/download, reopen the terminal, and verify:

```
node --version
npm --version
```

> **Workspace trust:** trust only a repository whose source you know. Read its README and scripts before running commands. Do not disable security controls or run unexplained administrator commands to make an error disappear.

04 / GITHUB ACCESS

An account is not an invitation.

1. Secure your own GitHub account

Create or sign in to your account at github.com. Verify your email and enable two-factor authentication under Settings → Password and authentication. Store recovery codes privately, outside repositories and screenshots.

2. Request the correct private repository

Submit your GitHub username through the WIL portal's GitHub access request option. A username is not your password or email address. Staff must review the request; approved WIL membership does not automatically grant GitHub access.

3. Accept and verify the invitation

Check your GitHub notifications and invitation email. Sign in to the same account whose username you submitted. Accept the invitation, open the repository and confirm its owner/name and **Private** label. If the invitation has expired, request a fresh one.

4. Clone, do not download a ZIP

On the repository page select Code → HTTPS and copy its URL. In your terminal, move to your lab folder. Replace the placeholder URL with the invited repository's real URL:

```
cd PATH_TO_YOUR_LAB_FOLDER
git clone https://github.com/OWNER/REPOSITORY.git
cd REPOSITORY
git status
```

Pass: files are present and git status identifies a branch with no pending changes. Open this repository folder in VS Code, not its parent.

> **Authentication:** GitHub account passwords do not authenticate Git over HTTPS. Use a supported browser/credential-manager sign-in or GitHub Desktop. If a token is genuinely needed, follow official guidance and use the least required access; never paste it into a remote URL, source file, chat or screenshot.

Practice without staff access

You may create a separate **private** personal practice repository with a README. It does not replace the assigned repository or grant access to company work.

05 / COMMAND-LINE WORKFLOW

Your first traceable change.

Use a personal practice repository first, or an approved branch in the assigned repository. Do not alter another learner’s work. Replace the branch suffix with your own short username.

1. Start from an up-to-date, clean working tree

```
git status
git pull --ff-only
git switch -c setup/YOUR_USERNAME
```

If git status shows unfinished changes, stop and understand them before pulling. On a shared project, follow its branching rules; do not push straight to a protected branch.

2. Add a useful note

Create **setup-notes.md** in VS Code. Record the date, operating system, installed tool versions, checks performed and one limitation. Use synthetic examples, not private data. Save the file.

3. Inspect and commit only intended changes

```
git status
git diff
git add setup-notes.md
git diff --cached
git commit -m "docs: record local setup checks"
```

Read the staged content before committing. Prefer naming files explicitly rather than staging every file without inspection.

4. Push and read the result back

```
git push -u origin setup/YOUR_USERNAME
```

Open GitHub, select your branch and confirm your file and commit appear. A local commit alone is not a successful remote submission.

> **Pull request:** if the project requires review, use Compare & pull request. Check the base branch, describe what changed and your checks, and request the designated reviewer. Do not merge your own work unless authorised.

06 / OPTIONAL GRAPHICAL ROUTE

The same process in Desktop.

GitHub Desktop is optional for supported Windows and macOS computers. It performs Git operations; it does not replace understanding branches, commits or evidence. Linux learners should use page 5.

1. Install and sign in

Download from desktop.github.com. Sign in using the browser prompt and your own GitHub account. Do not give your password or recovery codes to a mentor or classmate.

2. Clone the correct repository

Choose File → Clone repository. Select the repository you were invited to, or use its HTTPS URL. Choose your dedicated local lab folder. If it is missing, check invitation acceptance and account identity before reinstalling software.

3. Branch, edit and inspect

Fetch origin. With a clean working tree, pull available changes. Under Current Branch create your personal setup branch, following the team’s rules. Open the repository in VS Code and write setup-notes.md. Return to Desktop and inspect the Changes list and each file’s difference.

4. Commit and publish

Select only the intended file. Enter a meaningful summary, commit to your branch, then Publish branch or Push origin, whichever is shown. Open the repository on GitHub and verify the remote branch contains your file.

5. Request review where required

Open a pull request from your branch to the approved base branch. Include what changed, how you checked it, and any remaining problem. Check the Files changed tab for accidental personal information.

> **Checkpoint:** explain the difference between saving a file, committing locally, and pushing to GitHub. If you cannot distinguish these, repeat a small change and observe each stage. Use either the command-line or Desktop route for your first practice, not both at once.

07 / LOCAL PRACTICE

Build something you understand.

Exercise: a small, learner-authored local page

In your private practice repository, create **index.html**. Type the example, then change the heading and paragraph to explain your own setup. This is a practice starting point, not a final-year project or assessed solution.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>My WIL setup check</title>
  </head>
  <body>
    <h1>My local setup works</h1>
    <p>I can edit, save and verify a local page.</p>
  </body>
</html>
```

Test before committing

• Save the file and open it in your browser using the file manager. A file:// address is expected for this exercise; it is not a hosted website or a localhost server.

• Change one sentence, save again and refresh the browser. Verify that the visible result changes.

• Inspect the difference in Git. Stage index.html, commit with a clear message, and push your practice branch. Verify the updated file on GitHub.

Record your result

Write a short note: what you changed, how you tested it, the observed result, and what you would improve. Capture a safe screenshot of the page. Keep your repository private.

> **For an assigned application:** follow its README and specified runtime/package manager. Read package scripts before installing dependencies or running commands. A browser opening this HTML file does not prove that a Node application, API or database is running.

08 / STREAM-SPECIFIC EVIDENCE

Show what actually happened.

Software development

Confirm your assigned stack from the project README or mentor instructions. Keep requirements, design notes and learner-authored code traceable through small commits. Evidence may include the private repository/PR link, test output, safe screenshots and an explanation of your contribution. Do not substitute the practice HTML page for assigned project requirements.

Cloud administration

Provider access remains deferred. Continue the existing task using local practice: synthetic-file inventories, permissions checks, backup/restore tests, local service logs and configuration proposals. Use a dedicated folder or an isolated VM if available; a VM is not required for every exercise.

> **Cloud reporting:** separate observed local results from simulated alerts and proposed IAM, MFA, budgets, firewall or provider settings. Upload screenshots and a PDF/DOC/DOCX report through the existing task. Include scripts/configuration as private repository links where relevant. No paid account, provider login or company administrator credentials are required.

Cybersecurity

Use synthetic data and systems you own or have explicit permission to test. Practise risk analysis, permissions review, policy drafting and training materials. Keep tests inside an isolated local lab. Do not scan public targets, attack campus/shared networks or disable protection on a shared device.

Submission and sign-off remain separate

For project submissions, provide the requested repository URL in the portal; do not upload connector credentials or make the repository public for a reviewer. Submit other evidence in the existing task and explain results and limitations. Request appropriate reviewer access if needed.

Complete your student profile, required documents—particularly certified ID—and daily/weekly work logs. Work will not be signed off while these are incomplete, but you can continue task submissions and portal access. Record actual work and time; a setup problem does not justify invented hours.

09 / TROUBLESHOOT METHODICALLY

Diagnose before you reset.

Use the sequence: reproduce → read the error → check the likely cause → change one thing → repeat the same check → record the result. Do not delete your repository or reinstall everything as your first response.

“git/node is not recognised” or “command not found”

Check installation completed, then reopen your terminal and editor. Run the version check again. Verify the installer’s PATH guidance for your operating system. Ask a lab administrator if installation permissions are restricted.

“Repository not found”, 404 or permission denied

Confirm the repository URL, accepted invitation and signed-in account. A private repository may appear missing when access is absent. Do not change repository visibility. GitHub portal approval and GitHub invitation acceptance are separate.

“Not a git repository”

You are probably in the wrong folder or a downloaded ZIP. Open the cloned repository folder and repeat git status. Do not run git init inside an assigned repository just to silence the error.

Push rejected / non-fast-forward / protected branch

Run git status and read the rejection. Preserve your changes. If clean and permitted, fetch and update using the team’s workflow. Use a personal branch and pull request for protected branches. Do not force-push. If there is a conflict, ask for targeted help before overwriting anything.

Application will not run / port already in use

Read the README, verify the required runtime and use the correct project folder. Inspect the first meaningful error, not only the last line. If a local port is occupied, stop only a process you recognise and own, or use the documented alternate-port option.

> **If a secret was committed:** stop sharing it and notify the mentor privately. Revoke/rotate the credential through its owner first. Deleting the file in a later commit does not remove the secret from Git history. Follow authorised cleanup guidance; do not rewrite a shared history on your own.

10 / ASK FOR TARGETED HELP

Bring evidence of your attempt.

Before requesting help

• Attempt the relevant steps and official troubleshooting documentation. Keep a short record of what you tried.

• Reproduce the problem once and capture the exact error, removing tokens, personal paths, emails and unrelated browser tabs.

• Avoid screenshots of account security pages, recovery codes, .env files or certified ID documents.

Support request template

Subject: WIL setup support — [stream] — [blocked step]
GitHub username: [your username; no password]
Device: [operating system/version; managed or personal]
Task/repository: [private reference; share only with authorised staff]
Expected result: [what should happen]
Actual result: [exact sanitised error]
Steps to reproduce: [numbered actions]
Checks attempted: [steps and observed outcomes]
Evidence: [safe screenshot or authorised private link]
Current limitation: [installation, device, network or access issue]

Where to send it

Use the cohort’s established support channel, or contact info@plkinchub.co.za. Do not send personal documents or secrets with the setup request. A mentor may request a specific additional check rather than taking over your machine.

While you are blocked

Continue suitable documentation, planning or local practice within your current task. Record the blocker and actual work in your logs. Keep existing submissions and deadlines; ask your mentor if the blocker affects a required outcome. Setup support does not automatically approve or sign off work.

11 / REFERENCES & SELF-CHECK

Read the source. Explain the result.

Official documentation

Git downloads
https://git-scm.com/downloads

VS Code setup
https://code.visualstudio.com/docs/setup/setup-overview

VS Code workspace trust
https://code.visualstudio.com/docs/editing/workspaces/workspace-trust

GitHub repository cloning
https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

GitHub two-factor authentication
https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa

GitHub credential management
https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git

GitHub Desktop
https://docs.github.com/en/desktop

Node.js downloads (only if required)
https://nodejs.org/en/download

Explain without looking at the guide

- What is the difference between Git, GitHub and VS Code?
- How do you prove a local commit reached the correct remote branch?
- Why does a private repository require an invitation even if you can log in?
- What belongs in a work log, and what must never appear in a repository?
- For Cloud evidence, what is observed locally, what is simulated, and what is only proposed?

Tool interfaces and supported versions change. Follow current official instructions and the assigned repository's README if labels differ. This handbook adds setup guidance only; it does not change the programme, access policies, assessment requirements or deadlines.